

REST API PROJECT DOCUMENTATION

1. Introduction

This project is a REST API built using Flask and SQLAlchemy to manage videos and track user interactions such as likes and views.

The system follows proper database normalization by using foreign key relationships instead of storing redundant data.

2. Technologies Used

- Python
- Flask
- Flask-RESTful
- SQLAlchemy
- PostgreSQL
- Postman (for testing)

3. Database Design

a. Video Table (`video_model`)

Column	Type	Description
id	Integer	Primary Key
name	String	Video name
is_deleted	Boolean	Soft delete flag
deleted_at	DateTime	Deletion timestamp

b. Like Table (`like`)

Column	Type	Description
id	Integer	Primary Key
video_id	Integer	Foreign key to video_model
user_name	String	User who liked

c. View Table (`view`)

Column	Type	Description
id	Integer	Primary Key
video_id	Integer	Foreign key to video_model
viewed_at	DateTime	Timestamp

4. Database Relationships

video_model (1)
↓
like (many)
view (many)

5. API Endpoints

A. Video APIs:

Create Video: PUT /video/<id>

body:

```
{  
  "name": "test1"  
}
```

409 conflict { "message": "Video already exists" }	201 created { "id": 4, "name": "test1" }
--	---

Get Video : GET /video/<id>

200 OK { "id": 1, "name": "test1", "likes": 3, "views": 5 }	404 Not Found { "message": "Video not found" }
--	--

Update Video: PATCH /video/<id>

body:

```
{  
  "name": "test5"  
}
```

200 OK { "id": 1, "name": "test5" }	404 Not Found { "message": "Video not found" }
--	--

Soft Delete: DELETE /video/<id>

200 OK

```
{
  "message": "Soft deleted"
}
```

B .Like API:

Add Like: POST /video/<id>/like

body:

```
{
  "user_name": "parthi"
}
```

200 OK

```
{
  "message": "Liked"
}
```

200 OK

```
{
  "message": "Already liked"
}
```

C. View API:

Add View: POST /video/<id>/view

201 created

```
{
  "message": "Viewed"
}
```

6. Testing

APIs were tested using **Postman**:

- Create video
- Add likes
- Add views
- Fetch video data

7. Conclusion

This project demonstrates:

- REST API design
- Foreign key relationships
- Data normalization
- Soft delete strategy
- It follows industry-level backend design principles.

